TECHNICAL PROJECT REPORT

INT4 KV-Cache Quantization with Fused Flash-Decoding Attention

One decode kernel, three backends: CUDA, Triton, and Pallas/JAX

Author	Archana Suresh Patil
Date	27 August 2026
Repository	`github.com/ArchanaChetan07/int4-kv-cache-quantization-cuda-triton-pallas`
License	Apache-2.0
Test suite	76 passing with JAX, 53 without; CI green on 5 jobs

WHAT THIS DOCUMENT IS

A complete account of a kernel-engineering project: what it computes, how it is implemented three times over, what was measured, what was learned from porting between two kernel-programming models, and -- stated as plainly as the results -- what has NOT been established.

01

Executive summary

The problem

Serving a large language model is dominated by memory, not arithmetic. During decoding, every generated token must read the entire key-value (KV) cache for every attention head. That cache grows linearly with sequence length and batch size, and reading it saturates memory bandwidth long before the GPU runs out of compute. The KV cache is therefore the binding constraint on how many concurrent requests a server can hold and how fast it can answer them.

The approach

Store the keys as 4-bit integers instead of 16-bit floats -- a 4x reduction -- and fuse the dequantization into the attention kernel so the full-precision key matrix is never written to memory at all. Combine this with flash decoding, which streams the online softmax over paged KV blocks and never materializes the attention matrix. The result moves roughly a quarter of the bytes.

What makes this project unusual

The same algorithm is implemented three times -- in CUDA C++, in Triton, and in JAX/Pallas -- and all three are validated against a single NumPy reference. Holding the algorithm and the oracle fixed turns a re-implementation into a controlled experiment: any difficulty encountered is attributable to the tool, not to the mathematics. The Pallas port was the experiment, and the findings in section 06 are its result.

Measured outcome	Result
Quantizer vs NumPy oracle (CUDA and Pallas)	**0.000% bin disagreement**
KV memory compression vs FP16, incl. scale/zero-point overhead	**3.98x** measured
Fused attention -- agreement with FP32 reference (head_dim 64)	MAE **3.1e-08**
Kernel latency, batch 8 x 32 heads x dim 128 x seq 2048	**2.84 ms**, 3.9x over a serial baseline (T1000)
Pallas attention error vs a float64 evaluation	within **1.4x** of the float32 accumulation floor
Test suite	**76 passing** with JAX, 53 without; 5/5 CI jobs green

THE HONEST HEADLINE

Correctness is established on every backend. **Performance is not.** Pallas has no CPU code generator, and the available GPU is below the hardware floor of both live Pallas GPU backends, so every Pallas timing in this repository is interpret-mode and is correctness evidence rather than a performance number. Section 09 states the limits precisely.

02

What the kernel computes

Per-channel asymmetric INT4 quantization

Each channel of the key cache gets its own scale and zero-point, derived from that channel's observed range. Per-channel granularity matters: activation channels in transformers have wildly different dynamic ranges, and a single scale across all of them wastes most of the 16 available levels on the widest channel.

```
scale[c] = (max[c] - min[c]) / 15
zp[c] = -min[c] / scale[c]
q = clip(round((kv - min[c]) / scale[c]), 0, 15)
```

Sixteen levels means fifteen intervals, so the maximum round-trip error per value is half a scale step. That bound is asserted directly in the test suite rather than assumed.

Flash decoding with online softmax

A naive attention implementation computes all logits, takes a softmax, then weights the values -- which requires holding the full attention matrix. Online softmax instead keeps a running maximum and a running normalizer, rescaling the accumulated output each time a larger logit appears. Memory stays constant in sequence length.

```
m_new = max(m_old, max(logits))
corr = exp(m_old - m_new)
l_new = l_old * corr + sum(exp(logits - m_new))
o_new = o_old * corr + V @ exp(logits - m_new)
```

Fusing the two is where the memory saving is actually realized. Keys are read as INT4, dequantized in registers, consumed immediately by the dot product, and discarded. The FP32 key matrix exists only transiently inside the kernel.

WHY AN EMPTY PAGE IS DANGEROUS

A paged KV cache can contain pages with zero valid rows. The online-softmax update then computes exp(-inf - -inf), which is NaN and silently poisons the whole sequence. Every implementation here guards it, and the guard differs by backend -- see section 06.

03

Three implementations, one oracle

The NumPy reference is the single source of truth. It is deliberately slow, deliberately simple, and every backend is measured against it and nothing else.

Implementation	Covers	Parallel decomposition
NumPy reference quantize_int4_ref.py flash_decode_ref.py	quantizer + attention	none -- the definition of correct
CUDA C++ flash_decode_int4.cu	quantizer + attention	one thread block per (batch, head); warps stride over positions; warp-shuffle reduction; cross-warp log-sum-exp merge
Triton quantize_int4_triton.py	quantizer only	one program per channel, looping over rows inside the program
Pallas / JAX quantize_int4_pallas.py flash_decode_pallas.py	quantizer + attention	grid over (batch, heads, blocks); accumulators are resident output blocks carried across a sequential grid

There is no Triton attention kernel. That is a real gap and it is enforced in code: requesting backend='triton' for attention raises an error naming the gap rather than silently returning reference results. An earlier version of the dispatch layer did fall through silently -- see section 08.

Validating without an accelerator

Both kernel DSLs offer an interpreter that runs kernels as ordinary host code. Triton has TRITON_INTERPRET=1; Pallas has pallas_call(interpret=True). Continuous integration uses both, so every numerical claim is re-checked on every push using free CPU runners. This is what keeps a kernel repository honest when the author does not own the target hardware.

04

Why port a kernel you already wrote

The Pallas port exists to answer a question about tooling, not about attention. If an unfamiliar algorithm had been chosen, every obstacle would be ambiguous: is this hard because Pallas is hard, or because the mathematics is not yet understood? Holding the algorithm and the oracle fixed removes that ambiguity. Every hour of difficulty becomes attributable to the tool. That attribution is the deliverable.

To keep the exercise honest, fourteen predictions were written down before any code was ported -- which Triton habits would transfer, which would need restructuring, and which would actively mislead. They were scored afterwards, including the ones that were wrong. A retrospective written after the fact can always make its author look prescient; a pre-registered one cannot.

Verdict	Count	Meaning
CONFIRMED	10	the prediction held
REFUTED	1	scalar prefetch was predicted to be required for variable-length pages; a plain input specification worked. It is a performance mechanism, not a correctness requirement.
UNRESOLVED	3	all three require TPU silicon to test and are not asserted

THE MOST USEFUL REFRAMING

The project was framed as 'where the mental model transferred and where it misled'. The more useful axis turned out to be **what disappears**. Warp machinery, shared-memory staging, the cross-warp merge, occupancy tuning -- these are not translated into Pallas equivalents. They stop being things. A transition guide organized as 'here is the Pallas way to do X' is the wrong shape for half the work, because for that half X is no longer something you do.

05

The hardware constraint

A scoping check before writing code invalidated the obvious version of this project, and it is worth stating plainly because it shapes everything after it.

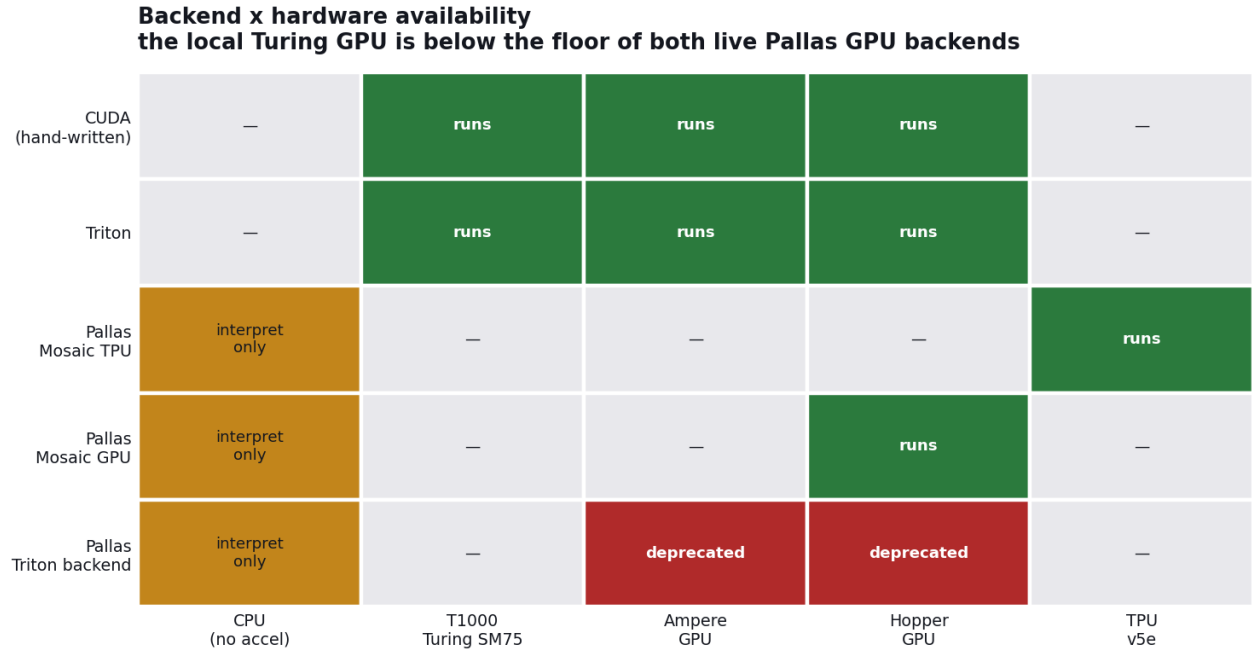

Figure 1. Which kernel backend runs on which silicon. Measured and documented, not estimated.

Pallas has two GPU backends. Mosaic GPU, the recommended one, targets Hopper and Blackwell. The Triton backend, which would otherwise have covered older cards, is deprecated with removal announced. The development machine is an NVIDIA T1000 -- Turing, compute capability 7.5 -- which sits below the floor of both.

Pallas also has no CPU code generator at all. Calling pallas_call with interpret=False on a CPU host raises 'Only interpret mode is supported on CPU backend'. There is no slow-but-real fallback.

CONSEQUENCE

Porting Triton to Pallas *on GPU* would have compiled the Pallas code back through Triton -- comparing a language against itself through a deprecated adapter. The port therefore targets Mosaic TPU, where there is no shared substrate and the comparison is real: no warps, no manual shared memory, no thread count, a fixed (8,128) register tiling, and a grid that runs sequentially rather than as a swarm of independent programs.

06

Four findings that required building it

6.1 The oracle is less accurate than the kernel it validates

Validating against the FP32 reference alone is insufficient. Measured against a float64 evaluation of the same mathematics, at head_dim 128 the reference's own error is 9.26e-07 while the Pallas kernel's is 2.59e-07. The reference is not broken -- it is simply sitting at the float32 accumulation floor, sqrt(D) * eps * |output|, which predicts 8.0e-07.

The consequence is sharp: a gate of the form 'agreement with the reference below 3e-08' is **unsatisfiable at that shape by any correct implementation**. Agreement with an FP32 oracle bounds nothing about accuracy. The attention tests therefore use a float64 arbiter and gate on the accumulation floor.

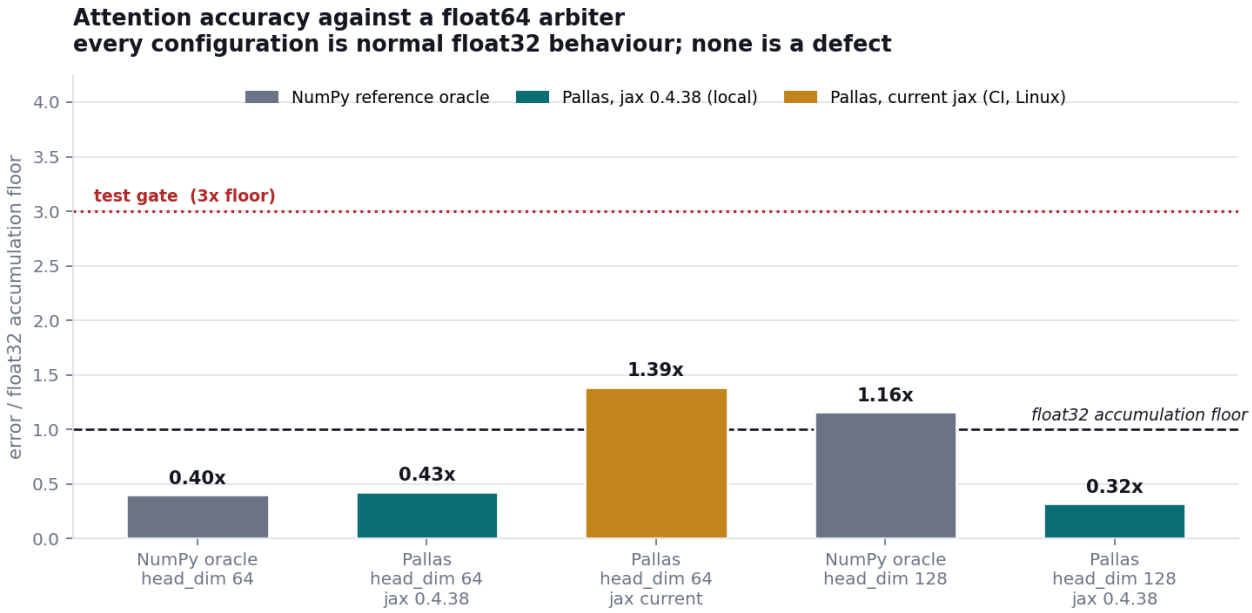

Figure 2. Error against a float64 arbiter, normalized by the float32 accumulation floor. Every configuration is normal float32 behaviour.

6.2 Which implementation is more accurate is a property of the compiler

The first version of that gate asserted the Pallas kernel was at least as accurate as the reference. It passed locally and failed in continuous integration. The reference is byte-identical on both platforms -- it is NumPy, and deterministic. Only the Pallas error moved, by 3.2x, because XLA is free to choose a different reduction order between versions.

So the ordering is not a property of either implementation; it is a property of the compiler build. The gate now asserts what is stable: both sit within 3x of the float32 floor. That still fails loudly for a real defect, which misses by orders of magnitude rather than by 1.4x. A tolerance never checked on a second toolchain is an assumption, not a measurement.

6.3 A correctness bug that only exists on GPU

The Pallas accumulators live in output blocks whose index map ignores the sequence axis, so Pallas keeps them resident and consecutive writes accumulate. This is safe only because the TPU grid executes sequentially in lexicographic order.

On GPU that guarantee does not hold: Mosaic GPU partitions dimensions marked 'parallel' across CUDA thread blocks, which would make the accumulator a race. Interpret mode cannot reveal this, because it executes the grid sequentially by construction. The fix declares the semantics explicitly -- (parallel, parallel, sequential) on GPU, (parallel, parallel, arbitrary) on TPU -- and was applied before any GPU time was purchased.

6.4 The same defect class, a different shape of fix

The CUDA kernel guards the empty-page NaN with a branch on negative infinity. Pallas has no cheap per-lane branch, and the fix is arithmetic instead: substitute a finite surrogate for the running maximum whenever it is negative infinity, which drives every exponential to exactly zero -- the correct contribution of an empty page. Same hazard, same reasoning, entirely different remedy. The CUDA instinct points at a construct that should not be reached for.

07

Benchmark methodology

The three backends cannot all run on one device, so the benchmark is built to make dishonest comparison structurally impossible rather than merely discouraged.

Rule	Rationale
Identical seeded inputs, byte for byte, across backends	removes input variation as an explanation for any gap
Milliseconds compared only within one device	two chips with different memory systems are not comparable in absolute time
Memory-bandwidth utilization is the cross-device metric	the kernel is bandwidth-bound, so MBU is what transfers between machines; where a device's peak bandwidth is not known, MBU is reported as null rather than guessed
Milliseconds compared only within one execution mode	interpret mode is a correctness vehicle, not a shipped code path; the harness records the mode and refuses to print a cross-mode ratio
Unavailable backends recorded with a reason	an absent row and a failed row must not look the same
Shape sweep fixed before any backend is timed	prevents choosing shapes after seeing results

Anti-cheat assertions

Any benchmark graded on wall-clock is gameable, including unintentionally by its author. Three assertions run inside the harness: that the output is actually consumed (a result nothing reads can be eliminated as dead code); that the kernel is not specialized to round numbers (every configuration is re-run at sequence length plus one); and that the kernel is present in the compiled output rather than having been replaced by a library call -- a kernel that gets optimized away benchmarks beautifully.

08

A defect worth reporting

A review of the dispatch layer found that requesting a backend which does not implement an operation returned results from a different backend, silently.

ops.flash_decode(backend='triton') returned NumPy reference results, byte-identical to backend='reference'. This repository has a Triton quantizer but no Triton attention kernel, and the dispatch chain fell through instead of refusing. Any misspelled backend name behaved the same way, for either operation.

WHY THIS ONE MATTERS MORE THAN IT LOOKS

Benchmarking that configuration would have measured NumPy and reported it as Triton. It is precisely the failure the harness anti-cheat assertions exist to catch -- sitting one layer beneath them, where they could not see it. The lesson is that integrity checks at the measurement layer do not protect against dishonesty in the layer that selects what gets measured.

Backends are now declared per operation and validated: a named backend is delivered or raises. Auto-detection is unchanged, so no existing caller is affected. Four further input-validation defects were fixed alongside it -- degenerate shapes raising ZeroDivisionError, a zero-iteration timing loop raising UnboundLocalError, and a misleading error that reported a missing CUDA extension when the extension was present but the input shape was unsupported. All have regression tests.

09

What has NOT been established

Stated as plainly as the results, because a report that only lists what worked is not a technical report.

Claim	Status
Pallas kernels are correct	**Established** in interpret mode, on two platforms, in CI
Pallas kernels compile on a real backend	**Not established.** They have never been lowered by Mosaic. Interpret mode enforces none of the constraints a real backend imposes.
Pallas kernels are fast	**Not established.** No compiled Pallas timing exists on any device.
Three-backend performance comparison	**Not established.** Requires one device that runs all three; no such device has been used.
INT4 KV preserves model quality	**Simulated only.** The perplexity gate has not run against real model weights.
Speedup versus a production kernel	**Not measured.** The 3.9x figure is against a serial baseline in this repository, not against FlashAttention or a serving engine.
TPU-specific behaviour (int4 dtype, packing axis, reduction direction)	**Unresolved.** Three pre-registered hypotheses require TPU silicon.

The distinction between the first row and the rest is the distinction between a kernel that is right and a kernel that is good. Only the first has been shown.

10

Next steps

Step	Cost	What it resolves
Rented H100 run -- tooling is built and tested: preflight, bootstrap, ordered runbook, isolated lowering probe	~$5, 1.5 h	all three backends on identical silicon; real datacenter numbers; the first compilation of these kernels; the real perplexity gate
TPU leg on free Colab or Kaggle	free	the three unresolved hypotheses, which an H100 cannot touch because they are Mosaic TPU properties
Baseline against PyTorch SDPA	free	replaces a speedup measured against this repository's own first draft with a defensible external comparison
Upstream compiler contribution -- an open Triton pull request awaiting review	free	external validation, which outweighs every self-reported number here

THE FORK TO PLAN FOR

The lowering probe has three outcomes and they are not equally good news. LOWERED means the port is proven. REJECTED means Mosaic refused the kernel -- keep the error text, it is the finding. MISMATCH means it compiled but disagreed with interpret mode, which is the signature of a grid-order race rather than a numerics problem, and must not be chased with tolerances.

Reproducing everything in this report

```
git clone https://github.com/ArchanaChetan07/int4-kv-cache-quantization-cuda-triton-pallas
pip install -e ".[pallas]"
pytest tests/ -q
python benchmarks/bench_three_backend.py --quick
python scripts/make_pallas_charts.py
python scripts/make_project_pdf.py
```

Every figure, table and number in this document is produced by the repository it describes. Nothing is illustrative.